FORM-PC-01: Comparación de modelos de proceso de requisitos en escenarios reales

Arteaga Alava Danela Dayana, Chavarría Cuenca Tahiny Mel,
Naranjo Alfonzo Anthony Alexander, Nivela Saavedra José Ricardo
Grupo ACNN · Carrera de Software
Facultad de Ciencias de la Computación
Universidad Técnica Estatal de Quevedo, Ecuador
Ingeniería de Requerimientos – ISR-401

Resumen

Actividad formativa grupal que compara los modelos de proceso de requisitos en cascada, iterativo y ágil en escenarios reales. Se definieron seis criterios de comparación (estabilidad de los requisitos, criticidad, disponibilidad del cliente, tamaño y complejidad, regulación y contrato, y documentación exigida) y con ellos se elaboró una matriz para elegir el modelo más adecuado en cada escenario. Para el sistema de un GAD con contrato público y para el sistema embebido de una planta se eligió el modelo en cascada, para la plataforma de una cooperativa de ahorro y crédito, el modelo iterativo, y para la aplicación móvil de un emprendimiento, el modelo ágil. Además, se aplicó el framework de Pohl al escenario del GAD, indicando las actividades centrales y transversales de cada fase y se identificaron los riesgos del modelo elegido en cada escenario.

Palabras clave

Ingeniería de requisitos, modelos de proceso, cascada, iterativo, ágil, framework de Pohl.

Datos generales

Campo	Detalle
Asignatura	Ingeniería de Requerimientos (ISR-401) · 4.º nivel · Paralelo B
Grupo	ACNN
Integrantes	Arteaga Alava Danela Dayana, Chavarría Cuenca Tahiny Mel, Naranjo Alfonzo Anthony Alexander, Nivela Saavedra José Ricardo
Docente	Gleiston Cicerón Guerrero Ulloa, PhD
Fecha de cierre	Sábado 7 de noviembre de 2026, 23:59
Repositorio	https://github.com/darteagaa-boop/ISR401-GrupoACNN

I. Criterios de comparación

Para comparar los modelos en cascada, iterativo y ágil en los cuatro escenarios se usan seis criterios evaluados cada uno según las características del escenario.

Tabla I
Criterios de comparación

Criterio	Qué evalúa	Valoración
C1. Estabilidad de los requisitos	Qué tan probable es que los requisitos cambien durante el proyecto.	Alta, media o baja
C2. Criticidad del sistema	Qué tan graves son las consecuencias de una falla, de dinero o de operación para las personas.	Alta, media o baja
C3. Disponibilidad del cliente	Con qué frecuencia el cliente y los usuarios pueden participar para aclarar, priorizar y validar los requisitos.	Alta, media o baja
C4. Tamaño y complejidad	Cantidad de funciones, actores, sistemas externos e integrantes del equipo involucrados.	Grande, mediano o pequeño
C5. Regulación y contrato	Si existen leyes, normas o un contrato que fijen el alcance, el plazo, el costo o la forma de aceptar el producto.	Alta, media o baja
C6. Documentación exigida	Cuánta documentación formal se requiere para aprobar, auditar o mantener el sistema.	Alta, media o baja

II. Caracterización de los modelos de proceso

II-A. Modelo en cascada

El modelo en cascada organiza el trabajo en fases sucesivas, con productos revisados y aprobados antes de avanzar a la siguiente etapa. En ingeniería de requisitos, su característica principal es establecer una especificación suficientemente completa y validada antes de usarla como base del diseño y la implementación. Esto facilita planificar entregables y conservar la trazabilidad, pero también puede hacer costoso descubrir requisitos omitidos en etapas tardías [1], [2].

II-B. Fases principales

1. **Delimitación y viabilidad.** Identificar el problema, los objetivos, las partes interesadas, el alcance, las restricciones y los riesgos iniciales. *Resultado:* alcance y plan de trabajo acordados.
2. **Obtención y análisis de requisitos.** Recopilar las necesidades de los actores, revisar el funcionamiento del entorno, resolver contradicciones y distinguir requisitos funcionales, de calidad y restricciones. *Resultado:* requisitos analizados y priorizados.
3. **Especificación y validación.** Documentar los requisitos con identificadores y criterios de aceptación; comprobar su coherencia, factibilidad, verificabilidad y correspondencia con las necesidades del cliente. Una vez aprobados, forman una *línea base* sometida a control de cambios. *Resultado:* especificación validada [2], [3].
4. **Diseño de la solución.** Definir arquitectura, módulos, interfaces y decisiones técnicas a partir de la especificación aprobada. *Resultado:* diseños revisados y relacionados con sus requisitos.
5. **Implementación e integración.** Construir los componentes e integrarlos respetando las interfaces acordadas. *Resultado:* versión integrada del sistema.
6. **Verificación, validación y aceptación.** Ejecutar pruebas de componentes, integración, sistema y aceptación; contrastar los resultados con requisitos y criterios previamente definidos. *Resultado:* evidencias de conformidad y registro de defectos.
7. **Despliegue y mantenimiento.** Instalar, operar, corregir defectos y tramitar nuevas necesidades mediante solicitudes de cambio evaluadas por su impacto. *Resultado:* sistema operativo con documentación y versiones controladas.

Las fases describen una secuencia predominante, no la prohibición absoluta de retroceder: una no conformidad puede exigir revisar una fase anterior, pero el cambio debe evaluarse y documentarse.

II-C. Supuestos del modelo

Para que esta organización sea razonable, se parte de los siguientes supuestos, que deben comprobarse en el proyecto concreto:

1. Las necesidades fundamentales pueden conocerse y aclararse antes de comprometer el diseño detallado.
2. Los requisitos centrales y las interfaces tendrán una estabilidad relativa durante la construcción.
3. Los responsables pueden revisar y aprobar entregables en hitos definidos, aunque no participen todos los días.
4. Es viable especificar criterios de aceptación y conservar evidencias de cumplimiento para cada requisito relevante.
5. Los cambios posteriores se gestionarán mediante un procedimiento explícito de análisis de impacto, aprobación y actualización documental.

II-D. Condiciones de aplicación y límites

Es favorable cuando el alcance está delimitado, existen interfaces técnicas conocidas, hay exigencias de documentación y trazabilidad, las revisiones formales son importantes y el cliente puede aprobar una línea base antes del desarrollo [1].

Es poco conveniente como proceso estrictamente secuencial cuando los objetivos cambian con frecuencia, se desconoce la viabilidad técnica, se necesita experimentación continua o las personas usuarias solo pueden aclarar sus expectativas al probar prototipos. En estos casos, conviene considerar un enfoque iterativo o introducir una exploración técnica previa a la aprobación de la línea base.

II-E. Modelo iterativo

El modelo iterativo plantea que los requisitos no tienen que quedar completamente resueltos en un único intento. Se comienza con una comprensión inicial del problema y se realizan ciclos de obtención, documentación, negociación y validación para corregir ambigüedades y ampliar la especificación. Cada ciclo aprovecha lo aprendido en el anterior; por eso es útil cuando existen necesidades conocidas, pero todavía quedan detalles por aclarar [2], [4].

A diferencia de una secuencia estrictamente lineal, una observación del cliente puede llevar a revisar requisitos ya documentados. El producto de cada ciclo puede ser una versión mejorada de la Especificación de Requisitos de Software (ERS), un prototipo evaluado o un conjunto de requisitos acordados. Iterar sobre requisitos no obliga, por sí solo, a entregar software funcionando en cada ciclo; esa decisión depende de cómo se organice el desarrollo.

II-E1. Fases principales:

1. **Comprensión inicial y delimitación.** Identificar el problema, los objetivos, los actores, las restricciones y los requisitos conocidos. También se seleccionan los aspectos con mayor incertidumbre. *Resultado:* alcance preliminar y primera versión de requisitos.
2. **Planificación de la iteración.** Elegir qué funcionalidades, dudas o riesgos se analizarán en el ciclo y acordar quiénes revisarán los resultados. *Resultado:* objetivos y criterios de revisión del ciclo.

3. **Obtención y documentación.** Realizar entrevistas, observaciones o revisión de prototipos para precisar necesidades; actualizar requisitos funcionales, de calidad y restricciones, identificando sus fuentes. *Resultado:* ERS o modelos revisados.
4. **Negociación y validación.** Contrastar los requisitos con las partes interesadas, resolver contradicciones, comprobar que sean claros y verificables, y registrar los acuerdos. *Resultado:* requisitos aceptados y observaciones pendientes [2], [3].
5. **Evaluación y siguiente ciclo.** Revisar las observaciones, controlar las versiones, actualizar las prioridades y decidir si se necesita otra iteración o si ya existe una especificación suficiente para continuar. *Resultado:* nueva línea base o cierre documentado.

La validación y la gestión de cambios acompañan a todo el proceso, no se reservan únicamente para el último paso. Esto mantiene la trazabilidad entre las necesidades del cliente, las versiones de los requisitos y las decisiones tomadas [2].

II-E2. Supuestos del modelo:

- Existen partes interesadas disponibles para revisar los resultados en momentos acordados y dar respuestas a las dudas importantes.
- Los requisitos pueden precisarse progresivamente sin que el costo de cada corrección haga inviable el proyecto.
- Es posible dividir el problema en conjuntos de requisitos o asuntos prioritarios que puedan analizarse y validarse por ciclos.
- El equipo dispone de mecanismos para registrar versiones, decisiones, dependencias y solicitudes de cambio, evitando perder acuerdos anteriores.
- Se puede reservar tiempo para la retroalimentación y el retrabajo que surjan al descubrir necesidades no previstas.

II-E3. Condiciones de aplicación y límites: **Es conveniente** cuando el alcance general se conoce, pero aún hay incertidumbre sobre pantallas, procesos, reglas de negocio o integración con otros sistemas. También ayuda cuando los usuarios pueden revisar prototipos o versiones intermedias y es necesario conservar una especificación formal que mejore en cada ciclo [4].

Es menos conveniente si no existe ningún representante capaz de validar avances, si las decisiones dependen de autorizaciones demasiado lentas o si el proyecto no dispone de tiempo para revisar el trabajo. En sistemas con interfaces críticas o restricciones contractuales, las iteraciones deben mantener controles de trazabilidad, aceptación y análisis de impacto; repetir ciclos no elimina esas obligaciones.

Diferencia frente al enfoque ágil. Ambos aceptan revisiones, pero el iterativo puede centrarse en refinar la ERS y validar prototipos mediante ciclos planificados. En el enfoque ágil, la elaboración de requisitos suele integrarse de forma continua con la construcción y las pruebas de incrementos pequeños del producto [5].

II-F. Modelo ágil

El enfoque ágil organiza el desarrollo de software de forma incremental y empírica, con participación continua de las partes interesadas para obtener retroalimentación y ajustar el producto. Según Schön y sus colaboradores, las actividades de ingeniería de requisitos (obtención, documentación, validación, negociación y gestión) no están claramente separadas: se repiten durante las iteraciones y la información se detalla conforme se necesita para el trabajo siguiente [5].

II-F1. Fases y ciclo de trabajo: En los métodos ágiles las secuencias de fases no son únicas y en este caso se usará la propuesta de Scrum escrita por Schwaber [6]:

1. **Preparación (Pregame):** Incluye planificación de la entrega de los elementos que se conocieron en el backlog con las estimaciones de tiempos y costos.
2. **Desarrollo (Game).** Considerando los requisitos, el tiempo, la calidad, el costo y las condiciones del entorno se desarrollan ciclos iterativos que permiten evolucionar al sistema.
3. **Cierre (Postgame).** Se prepara el producto para su entrega mediante tareas como integración, pruebas del sistema y documentación final.

En la fase de desarrollo, Schwaber describe cuatro actividades recurrentes dentro del sprint [6]:

- **Desarrollar (Develop):** analizar, diseñar, implementar, probar y documentar los cambios necesarios para atender los requisitos del backlog.
- **Preparar la versión ejecutable (Wrap):** reunir los cambios en una versión ejecutable que permita mostrar lo implementado.
- **Revisar (Review):** presentar el trabajo, evaluar el progreso, resolver problemas y revisar riesgos. Pueden identificarse nuevos elementos del backlog.
- **Ajustar (Adjust):** incorporar al producto la información y los cambios derivados de la revisión.

Esta descripción corresponde a Scrum y se usó como base debido a que Scrum es un marco de trabajo ágil, pero no es una secuencia universal en las metodologías ágiles.

II-F2. Supuestos: Turk, France y Rumpe identifican supuestos que permiten analizar la aplicabilidad de los procesos ágiles. Para esta comparación se consideran los siguientes [7]:

- **Visibilidad del progreso.** La entrega de software funcional funciona como una ventana para que el cliente analice los avances de manera más limpia sin la necesidad de leer los informes concurrentes, pero esta característica se ve limitada cuando el sistema no muestra su evolución mediante una interfaz gráfica como lo puede ser un sistema embebido.
- **División del trabajo en iteraciones cortas.** Se supone que el producto puede descomponerse en incrementos pequeños, desarrollables y demostrables en periodos breves; las dependencias fuertes entre subsistemas pueden dificultar esta organización.
- **Disponibilidad del cliente.** La interacción con el cliente es clave en desarrollo ágil y es necesaria una buena comunicación y disposición entre el equipo y el cliente, pero las responsabilidades habituales del cliente pueden limitar esa disponibilidad y, por consiguiente, retrasar la retroalimentación.
- **Comunicación directa.** Los autores identifican el supuesto de que la conversación cara a cara es la forma más productiva de comunicación entre clientes y desarrolladores, y entre los propios desarrolladores.

II-F3. Condiciones de aplicación: El enfoque ágil puede funcionar cuando el cliente tiene tiempo para colaborar, las dudas se resuelven oportunamente y el trabajo puede dividirse en incrementos que se puedan evaluar. Esa participación permite aclarar los requisitos y cambiar las prioridades según lo que se aprende durante el desarrollo [5], [7].

Si el cliente tiene poca disponibilidad o hay dependencias técnicas que impiden dividir el trabajo, será necesario adaptar la forma de trabajar. También puede ser más difícil coordinarse cuando el equipo está en distintos lugares. En sistemas sin interfaz humana, el avance puede mostrarse mediante simulaciones, pruebas de aceptación o revisiones, como señalan Turk y sus colaboradores [7]. Estas dificultades deben analizarse en cada escenario antes de decidir si conviene usar un enfoque ágil.

III. Análisis de los escenarios

III-A. Sistema de un GAD con contrato público

III-A1. Caracterización: El docente presentó el escenario sin detalle, por lo que se describen los supuestos usados para el análisis.

Tabla II
Caracterización del escenario del GAD

Aspecto	**Descripción del escenario**
Sistema	Se asume que es una plataforma web para que los ciudadanos realicen trámites municipales en línea, ya sean solicitudes, pagos de tasas y seguimiento de trámites, para que los funcionarios los atiendan.
Cliente	El GAD municipal, representado por un administrador del contrato designado por la institución.
Actores del proceso	Administrador del contrato y fiscalizador por parte del GAD, funcionarios de los departamentos que atienden trámites, ciudadanos usuarios, autoridades del GAD que aprueban el alcance, el equipo proveedor y los organismos de control del uso de fondos públicos.
Estabilidad de los requisitos (C1)	Alta, ya que se asume que los requisitos se fijan en los términos de referencia del proceso de contratación pública antes de firmar el contrato, y cualquier cambio requiere una modificación formal.
Criticidad (C2)	Media, porque una falla no pone en riesgo vidas, pero afecta la atención a los ciudadanos y el manejo de datos personales y pagos.
Disponibilidad del cliente (C3)	Baja, ya que los funcionarios tienen otras responsabilidades y las decisiones pasan por aprobaciones internas, por lo que las reuniones son programadas y poco frecuentes.
Tamaño y complejidad (C4)	Grande, porque involucra varios departamentos, muchos tipos de trámite y la integración con sistemas existentes del GAD.
Regulación y contrato (C5)	Alta, el proyecto se rige por un contrato público con alcance, plazo y presupuesto definidos, y debe cumplir la normativa de contratación pública y de protección de datos personales.
Documentación exigida (C6)	Alta, porque se necesitan especificaciones, actas de entrega y recepción e informes de fiscalización para pagar cada entregable y responder ante los organismos de control.

III-A2. Modelo elegido y justificación: **Modelo elegido:** Cascada

Se elige el modelo en cascada por las siguientes razones del escenario:

1. Los requisitos quedan fijados en los términos de referencia antes de firmar el contrato, y cualquier cambio exige una modificación formal. La cascada parte justamente de requisitos estables definidos al inicio.
2. El contrato público define alcance, plazo y presupuesto cerrados, y los pagos se hacen contra entregables aprobados por el fiscalizador. Las fases secuenciales de la cascada, con un entregable revisado al final de cada una, se ajustan a esa forma de aceptar el producto.
3. La disponibilidad del cliente es baja y la cascada concentra la participación de los funcionarios en la obtención y validación inicial de los requisitos, y no depende de reuniones frecuentes en cada iteración.

4. Se exige mucha documentación formal para pagar cada entregable y responder ante los organismos de control, la cascada produce una especificación completa y documentos de cada fase que sirven como evidencia.

Modelos descartados:

Ágil: Supone requisitos cambiantes y un cliente disponible en cada iteración para priorizar y validar, condiciones que no se cumplen en el GAD. Además, su documentación mínima no alcanza para las actas y los informes que exige el contrato.

Iterativo: Se adapta mejor al tamaño del proyecto, porque permite entregar por partes la integración de varios departamentos. Sin embargo, revisar los requisitos en cada iteración choca con un alcance fijado por contrato y con un cliente poco disponible, por lo que obtiene menor total que la cascada en la matriz.

III-A3. Riesgos del modelo elegido y mitigación: Elegir la cascada en el GAD trae riesgos propios del modelo, sobre todo porque los errores en los requisitos se descubren tarde, por eso los principales riesgos y su mitigación son:

Tabla III: Riesgos del modelo en cascada en el GAD y su mitigación

Riesgo	Mitigación
Requisitos mal entendidos en los términos de referencia que se detectan recién en las pruebas o en la recepción, cuando corregirlos es más costoso.	Validar la especificación con prototipos de pantallas antes de cerrar la fase de requisitos y firmar un acta de aprobación con el fiscalizador.
Cambios en normativas municipales durante el proyecto que obligan a modificar requisitos ya aprobados.	Definir desde el inicio un procedimiento de control de cambios con análisis de impacto en costo y plazo, ligado a la cláusula de modificación del contrato.
Retrasos en la aprobación de entregables por la baja disponibilidad de los funcionarios.	Acordar al inicio un calendario de revisiones y designar un responsable por departamento con autoridad para aprobar.
Poca participación de los ciudadanos, que son los usuarios finales, lo que puede causar baja aceptación del sistema.	Incluir una encuesta a ciudadanos en la obtención y pruebas de usabilidad con ciudadanos en la fase de pruebas.
Integraciones con sistemas existentes del GAD que se descubren tarde y no estaban en la especificación.	Hacer un inventario de sistemas e interfaces del GAD durante la obtención y validar la integración en la fase de diseño.

III-B. Sistema embebido de una planta

Alcance del caso. Se asume que una planta industrial dedicada a la fabricación y envasado de productos necesita implementar un sistema embebido que permita automatizar y supervisar el funcionamiento de su línea de producción. Actualmente, algunos procesos requieren intervención manual, lo que puede ocasionar retrasos, errores operativos, desperdicio de materiales y dificultades para detectar fallos en la maquinaria. El sistema estará basado en un controlador embebido industrial que recibirá información de sensores de temperatura, presión, proximidad y nivel, con el propósito de monitorear las condiciones de operación en tiempo real. A partir de estos datos, el controlador accionará automáticamente equipos como motores, bandas transportadoras, bombas y válvulas, siguiendo una secuencia de funcionamiento previamente establecida. Asimismo, contará con una interfaz de supervisión que permitirá a los operarios visualizar el estado de los equipos, consultar parámetros de funcionamiento y recibir alertas ante situaciones anormales. Se contempla el registro de eventos y la ejecución de acciones de protección, como detener de manera controlada determinados equipos cuando se detecten fallos.

Límites del alcance: El proyecto se limita al monitoreo, control y automatización de la línea de producción, sin incluir procesos administrativos, comerciales o financieros de la planta. Se asume que los requisitos principales de funcionamiento y seguridad serán definidos antes del desarrollo, debido a la criticidad de las operaciones industriales.

III-B1. Caracterización: **Cliente, actores y características del proyecto**

- **Cliente y objetivo.** Se asume que el cliente es la administración de la planta y que busca supervisar la operación de una línea productiva, reducir fallos de operación y registrar estados de los equipos.
- **Actores.** Se asume la participación de una jefatura de producción (acepta objetivos operativos), operadores (usan las señales y alarmas), personal de mantenimiento (diagnostica fallos), especialistas en automatización (integran sensores y actuadores) y responsables de seguridad industrial (revisan peligros y medidas de protección).
- **Estabilidad de requisitos. Alta en el núcleo, bajo supuesto:** se presume que la función de control y las interfaces físicas pueden definirse antes de construir. Los ajustes de parámetros o la presentación de alarmas podrían cambiar y requieren gestión de cambios.
- **Criticidad. Alta, bajo supuesto:** una orden incorrecta a un actuador o una lectura defectuosa podría afectar la continuidad del proceso, provocar daños en equipos o generar peligros. Esta valoración exige revisar y probar los comportamientos ante fallos; no presupone que todo el software cumpla una función de seguridad certificada.
- **Contrato y regulación.** Se asume un contrato o acuerdo técnico con alcance, interfaces y criterios de aceptación definidos. La normativa obligatoria **no está indicada en el enunciado** y deberá determinarse según país, maquinaria y funciones implementadas. Si hay funciones eléctricas/electrónicas/programables relacionadas con la

seguridad, corresponde evaluar la pertinencia de IEC 61508-1:2010, sin afirmar que sea automáticamente obligatoria [8].

- **Disponibilidad del cliente. Limitada, bajo supuesto:** se presume que operadores y técnicos trabajan por turnos. Serían posibles reuniones programadas para aclarar requisitos, revisar documentos y participar en pruebas de aceptación, pero no necesariamente reuniones diarias.
- **Tamaño e integración. Medio, bajo supuesto:** el alcance incluye firmware o lógica de control, sensores, actuadores, una interfaz de operación y pruebas de integración con el equipo físico. No se dispone de estimaciones reales de esfuerzo ni de componentes.

III-B2. Modelo elegido y justificación: **Elección propuesta: modelo en cascada con verificaciones técnicas tempranas y control de cambios.** La elección solo se mantiene si la revisión inicial confirma los supuestos anteriores. Se justifica por:

1. **Estabilidad del núcleo de control:** permitiría acordar previamente qué señales se leen, qué acciones se autorizan y cuáles son los límites de operación, disminuyendo cambios tardíos en interfaces.
2. **Criticidad y trazabilidad:** permite establecer requisitos y criterios de prueba identificables, relacionarlos con el diseño y conservar actas y resultados para la aceptación.
3. **Participación del cliente por hitos:** se adapta mejor a revisiones programadas con producción, mantenimiento y seguridad industrial que a la exigencia de decisiones operativas diarias.

Por qué no el iterativo como modelo principal: sería preferible si existieran dudas relevantes sobre sensores, interfaces o respuestas del controlador que solo pudieran resolverse mediante prototipos. Bajo los supuestos actuales, esa incertidumbre se considera manejable con ensayos preliminares, sin redefinir todo el proceso por ciclos.

Por qué no el ágil como modelo principal: las posibles ventanas reducidas de acceso a la planta y la necesidad de acordar previamente las interfaces y evidencias de aceptación hacen menos conveniente depender de retroalimentación continua. Esto no significa que las prácticas ágiles sean incompatibles con sistemas embebidos.

III-B3. Riesgos del modelo elegido y mitigación:

1. **Requisitos incompletos descubiertos en pruebas finales.** *Mitigación:* revisar escenarios normales y de fallo con operadores, mantenimiento y seguridad antes de aprobar la especificación; usar una matriz de trazabilidad requisito–prueba.
2. **Incompatibilidades con sensores, actuadores o tiempos de respuesta.** *Mitigación:* comprobar interfaces y restricciones de hardware mediante ensayos de factibilidad antes de congelar el diseño; repetir pruebas de integración con los equipos disponibles.
3. **Fallos con consecuencias para personas o equipos.** *Mitigación:* realizar análisis de peligros, definir estados seguros y criterios verificables, probar condiciones anormales y someter las funciones de protección a revisión especializada. No confiar una parada de emergencia a una solución exclusivamente de software sin la evaluación de seguridad que corresponda.
4. **Cambios de alcance durante la construcción.** *Mitigación:* registrar la solicitud, estudiar efectos sobre firmware, electrónica, pruebas, costo y plazo, y aprobar una nueva versión de los requisitos antes de modificar la línea base.
5. **Acceso limitado a la planta y retraso de la aceptación.** *Mitigación:* acordar desde el inicio responsables, fechas de revisión, condiciones para acceder a equipos y un entorno de ensayo alternativo cuando sea viable.

III-C. Aplicación móvil de un emprendimiento

III-C1. Caracterización: El enunciado únicamente indica una aplicación móvil de un emprendimiento, sin precisar qué vende, su presupuesto o las condiciones de desarrollo. Por ello, se plantea un **escenario hipotético**: un pequeño emprendimiento de productos artesanales quiere ofrecer su catálogo y recibir pedidos desde una aplicación móvil. Los datos siguientes son supuestos de análisis, no resultados de entrevistas con un negocio real.

III-C1a. Descripción y alcance: Se asume que la aplicación permitirá a los clientes consultar productos, ver precios y disponibilidad, preparar una solicitud de pedido y conocer su estado. El propietario podrá actualizar el catálogo y confirmar o rechazar pedidos. En la primera versión, el pago y la entrega se coordinarán fuera de la aplicación. No se incluirán pasarela de pagos, facturación electrónica, servicio propio de reparto ni integración con sistemas empresariales.

III-C1b. Cliente y actores del proceso: El cliente es la persona propietaria del emprendimiento, quien establece las prioridades y acepta las funcionalidades. Los compradores son usuarios principales y ayudan a evaluar la facilidad de uso. Se asume que una persona encargada de atender pedidos participa en la definición del flujo de confirmación; en un negocio pequeño puede ser la misma propietaria. El equipo de desarrollo obtiene, documenta, negocia y valida los requisitos.

III-C1c. C1. Estabilidad de los requisitos: baja: Aunque el objetivo de mostrar productos y recibir pedidos está claro, se supone que la organización del catálogo, el proceso de compra, las notificaciones y la presentación de las pantallas cambiarán después de que el propietario y los clientes prueben las primeras versiones.

III-C1d. C2. Criticidad del sistema: baja: Una falla podría provocar pedidos perdidos, molestias a los clientes o una venta no concretada, pero el escenario no incluye control de equipos peligrosos ni procesamiento de pagos dentro de la aplicación. La valoración baja no elimina la necesidad de proteger los datos de contacto y verificar que cada pedido quede registrado correctamente.

III-C1e. C3. Disponibilidad del cliente: alta: Se asume que la persona propietaria puede responder consultas por mensajería durante la semana y revisar una demostración breve al cerrar cada ciclo. Se programarán sesiones puntuales con compradores para recibir opiniones sobre el catálogo y la solicitud de pedidos.

III-C1f. C4. Tamaño y complejidad: pequeño: El alcance inicial se limita al catálogo, los pedidos, su estado y la gestión básica por parte del propietario. Participan pocos tipos de usuario y no se contemplan integraciones externas complejas. Se supone un equipo de desarrollo reducido.

III-C1g. C5. Regulación y contrato: baja: Se asume que el proyecto no depende de una contratación pública ni de un contrato que congele todas las funciones desde el inicio. Las prioridades podrán acordarse directamente con la persona propietaria. Esto no significa que no existan obligaciones legales: antes de operar habrá que revisar las que correspondan al tratamiento de datos personales y a la actividad comercial del emprendimiento.

III-C1h. C6. Documentación exigida: baja: Se considera suficiente mantener historias de usuario, criterios de aceptación, un registro de decisiones, evidencias básicas de prueba y una guía de uso. No se supone una auditoría contractual que requiera informes extensos para aceptar cada función.

Tabla IV
Caracterización del escenario hipotético de la aplicación móvil

Criterio	**Valoración**
C1. Estabilidad de los requisitos	Baja
C2. Criticidad del sistema	Baja
C3. Disponibilidad del cliente	Alta
C4. Tamaño y complejidad	Pequeño
C5. Regulación y contrato	Baja
C6. Documentación exigida	Baja

III-C2. Modelo elegido y justificación: **Modelo elegido: ágil, con entregas incrementales y revisiones frecuentes.** La matriz de la Tabla VIII le asigna 18 puntos, frente a 14 del iterativo y 9 de cascada. La selección depende de que se cumpla la disponibilidad del propietario planteada como supuesto.

1. **Requisitos cambiantes:** las pantallas y el recorrido para hacer pedidos pueden ajustarse cuando los compradores usen prototipos o versiones iniciales. El enfoque ágil permite revisar y priorizar esas necesidades continuamente [5].
2. **Participación del propietario:** las revisiones breves y las respuestas frecuentes permiten decidir rápidamente qué mejorar o qué función tiene más valor para el negocio.
3. **Alcance pequeño y divisible:** se puede comenzar mostrando un catálogo funcional, añadir la solicitud de pedidos y luego incorporar la consulta de estados. Cada entrega permite observar problemas reales sin esperar a terminar toda la aplicación.
4. **Pocas restricciones contractuales y documentales:** el emprendimiento puede reajustar prioridades sin tramitar modificaciones formales extensas, manteniendo de todas maneras criterios de aceptación, pruebas y registro de cambios.

Por qué no cascada como modelo principal. Aunque facilitaría establecer de antemano una especificación y un plan, exigiría decidir desde el inicio detalles de experiencia de usuario que todavía no se conocen. Si las pruebas con clientes muestran que el flujo de pedidos es confuso, corregirlo tardíamente podría causar retrabajo.

Por qué no el iterativo como modelo principal. También es una alternativa válida, ya que permite refinar requisitos y evaluar prototipos. Sin embargo, bajo los supuestos de alta disponibilidad del propietario y funciones pequeñas que pueden ponerse a prueba, resulta más conveniente integrar las revisiones de requisitos con la construcción y entrega frecuente de funcionalidades. El iterativo sería una opción más fuerte si las revisiones solo pudieran efectuarse en hitos separados o se exigiera una ERS formal actualizada en cada ciclo [4], [5].

III-C3. Riesgos del modelo elegido y mitigación: Los riesgos siguientes corresponden al escenario supuesto; no son problemas comprobados en un emprendimiento real.

1. **Crecimiento continuo del alcance.** El propietario podría solicitar pagos en línea, reparto propio u otras funciones fuera del objetivo inicial. *Mitigación:* mantener una lista priorizada de historias de usuario, separar el alcance inicial de las mejoras futuras y revisar el costo y el tiempo antes de aceptar cambios relevantes.

2. **Requisitos confusos o decisiones contradictorias.** Al recibir comentarios frecuentes, el equipo podría modificar pantallas sin criterios claros. *Mitigación:* definir criterios de aceptación para cada historia, registrar decisiones y validar el flujo de pedido con el propietario antes de cerrar una entrega.
3. **Pérdida de disponibilidad del cliente.** Aunque se supone alta participación, las actividades diarias del negocio pueden impedir revisar una versión. *Mitigación:* acordar de antemano reuniones breves, nombrar a quien pueda responder dudas y no dar por aprobada una función que depende de una decisión pendiente.
4. **Errores introducidos al agregar funciones.** Una nueva pantalla o actualización podría afectar la creación y el seguimiento de pedidos. *Mitigación:* realizar pruebas del flujo completo de pedido en cada entrega, comprobar el registro de estados y probar la aplicación en dispositivos de distintas características.
5. **Protección de datos y documentación descuidadas.** La rapidez de las entregas podría llevar a almacenar más información de la necesaria o a omitir registros de cambios. *Mitigación:* recoger solo datos indispensables, establecer permisos de acceso, verificar requisitos de privacidad y mantener historias, decisiones y resultados de prueba vinculados a cada versión.

III-D. Plataforma de una cooperativa

III-D1. Caracterización: El enunciado únicamente menciona una plataforma de una cooperativa, sin especificar sus funciones ni condiciones de desarrollo. Para realizar la comparación, se plantea un caso hipotético de una cooperativa de ahorro y crédito. Las características y restricciones descritas son supuestos de trabajo, no datos obtenidos de una organización real.

III-D1a. Descripción de la plataforma: Se supone que la cooperativa necesita una plataforma web para que sus socios consulten saldos y movimientos, realicen transferencias entre cuentas de la misma institución y presenten solicitudes de crédito. Los empleados podrán revisar las solicitudes y registrar su seguimiento; la aprobación del crédito continuará a cargo del personal autorizado.

La plataforma se conectará con un sistema existente que mantiene las cuentas y registra las operaciones. No sustituirá ese sistema ni calculará directamente los saldos. Quedan fuera del alcance inicial las transferencias a otras instituciones, la aplicación móvil y la aprobación automática de créditos.

III-D1b. Cliente y actores del proceso: El cliente será la cooperativa, representada por su gerencia. Se consideran los siguientes actores:

- **Gerencia:** acuerda el alcance, las prioridades y la aceptación del producto.
- **Responsables de operaciones y crédito:** explican los procedimientos, las reglas del negocio y las excepciones que debe contemplar la plataforma.
- **Socios:** aportan necesidades de uso y participan en la revisión de consultas, transferencias y solicitudes.
- **Responsable de tecnología:** proporciona información sobre el sistema existente y sus interfaces.
- **Responsable de seguridad y cumplimiento:** revisa las necesidades de protección de datos, permisos y registro de operaciones.
- **Equipo de desarrollo:** obtiene, documenta y analiza los requisitos, identifica conflictos y prepara los incrementos para su validación.

III-D1c. C1. Estabilidad de los requisitos: media: Se supone que las operaciones principales y las reglas para registrar movimientos ya están definidas en el sistema existente. Sin embargo, los detalles de las pantallas, el seguimiento de solicitudes y las notificaciones necesitan aclararse con empleados y socios. Por ello, existe una base estable, pero parte de los requisitos puede ajustarse durante las revisiones.

III-D1d. C2. Criticidad del sistema: alta: La plataforma permitirá acceder a información financiera e iniciar transferencias. Una falla podría exponer datos, duplicar una operación o mostrar información incorrecta. Por esta razón, el escenario requiere comprobar los permisos de acceso, la integridad de las operaciones y la correspondencia de lo mostrado con los registros del sistema existente.

III-D1e. C3. Disponibilidad del cliente: media: Se supone que la cooperativa designará a un representante de operaciones para una reunión semanal y para responder dudas entre reuniones. Los responsables de crédito, seguridad y tecnología participarán en revisiones programadas. Los socios estarán disponibles para sesiones puntuales de validación, pero no para colaborar diariamente.

Esta disponibilidad permite recibir retroalimentación periódica, aunque algunas decisiones tendrán que esperar la intervención del responsable correspondiente.

III-D1f. C4. Tamaño y complejidad: mediano: Se plantea un equipo de cinco personas para desarrollar una plataforma con tres grupos principales de funciones: consultas, transferencias internas y solicitudes de crédito. Participan usuarios con distintos permisos y existe una integración con el sistema de cuentas.

El alcance se considera mediano por estar limitado a esas funciones. No obstante, la integración y el manejo de operaciones financieras añaden complejidad técnica y requieren pruebas específicas.

III-D1g. C5. Regulación y contrato: alta: Como supuesto del caso, la cooperativa exigirá revisiones de seguridad y cumplimiento antes de autorizar el uso de funciones que manejen datos u operaciones financieras. Las obligaciones normativas concretas deberán identificarse y verificarse antes de establecer la línea base de requisitos.

También se supone un contrato con presupuesto y fecha de entrega acordados, organizado por entregables. Los cambios que afecten alcance, costo o plazo deberán registrarse y aprobarse. Estas restricciones exigen controlar las modificaciones, aunque permiten revisar los requisitos dentro de los límites acordados.

III-D1h. C6. Documentación exigida: alta: Se supone que la cooperativa solicitará una especificación de requisitos, reglas de negocio, criterios de aceptación, descripción de interfaces y registros de cambios. También exigirá trazabilidad entre los requisitos y sus pruebas, junto con evidencias de validación y aprobación de los entregables.

Esta documentación permitirá comprobar qué se acordó, por qué se modificó y con qué resultados se aceptó cada función.

Tabla V
Caracterización del escenario hipotético de la cooperativa

Criterio	**Valoración**
C1. Estabilidad de los requisitos	Media
C2. Criticidad del sistema	Alta
C3. Disponibilidad del cliente	Media
C4. Tamaño y complejidad	Mediano
C5. Regulación y contrato	Alta
C6. Documentación exigida	Alta

Estas valoraciones describen el escenario hipotético. Se utilizarán como base para comparar los modelos en cascada, iterativo y ágil, sin asumir de antemano que alguno es el más adecuado.

III-D2. Modelo elegido y justificación: **Modelo elegido: iterativo con desarrollo incremental y revisiones documentadas.**

Se propone construir versiones sucesivas, revisar los requisitos con la cooperativa y ampliar las funciones de forma controlada. Se distingue entre iterar, que permite revisar y mejorar lo trabajado, e incrementar, que añade funcionalidad. Ambas prácticas se combinan en la propuesta para este caso [4].

La elección se apoya en las siguientes razones del escenario:

1. **Hay requisitos que todavía necesitan aclararse.** Las operaciones principales tienen una base definida, pero las pantallas, las notificaciones y el seguimiento de solicitudes pueden cambiar tras las revisiones. Los ciclos permiten precisar esos detalles antes de extenderlos a toda la plataforma.
2. **La integración puede comprobarse progresivamente.** Se plantea comenzar con consultas de saldos y movimientos para revisar la conexión con el sistema existente, los permisos y la correspondencia de los datos. Después se incorporarían las transferencias internas y las solicitudes de crédito. Cada incremento necesitará pruebas y aprobación antes de ponerse en operación.
3. **La disponibilidad del cliente permite revisiones programadas.** La reunión semanal con operaciones puede utilizarse para aclarar dudas y revisar avances. Las decisiones de crédito, tecnología y seguridad se agendarán antes del cierre de cada ciclo. Se supone que estos responsables pueden cumplir esos compromisos; si no ocurre, deberá revisarse la planificación.
4. **La documentación puede mantenerse por incremento.** Antes de aceptar cada entrega se actualizarán los requisitos, las interfaces, los criterios de aceptación, la trazabilidad y los resultados de pruebas. Los cambios con impacto en alcance, costo o plazo se someterán a la aprobación prevista en el contrato supuesto.

La matriz de la Tabla IX respalda esta elección con 17 puntos. La ventaja sobre cascada es reducida y depende principalmente de poder aclarar requisitos e integrar el sistema de cuentas por partes sin abandonar las revisiones formales.

Alternativas descartadas para este escenario:

Cascada. Es una alternativa viable por el control de cambios, la documentación y las aprobaciones que exige la cooperativa. No se elige como organización principal porque todavía quedan requisitos de uso por precisar y conviene comprobar pronto la integración con el sistema existente. Esperar a una etapa posterior para validar estos aspectos aumentaría el retrabajo. Su valoración de 16 puntos muestra que sigue siendo una opción cercana, especialmente si se aclararan los requisitos pendientes antes de construir.

Ágil. También permite revisar requisitos y entregar incrementos, por lo que responde bien a la estabilidad media y al tamaño del proyecto. Se considera menos conveniente en las condiciones asumidas porque socios y especialistas no estarán disponibles de forma continua y deben coordinarse aprobaciones documentales y contractuales. El enfoque ágil

puede incorporar esos controles; su descarte no se basa en que carezca de documentación, sino en que la propuesta iterativa planificada se ajusta mejor a la participación programada y a los entregables formales del caso [5], [7].

III-D3. Riesgos del modelo elegido y mitigación: Los siguientes riesgos se plantean a partir de los supuestos de la cooperativa y de la propuesta de desarrollar la plataforma mediante ciclos e incrementos. No representan problemas observados en una organización real.

1. **Retraso de las revisiones.** Si operaciones, crédito o seguridad no revisan un incremento a tiempo, el siguiente ciclo podría comenzar con requisitos pendientes y generar retrabajo. **Mitigación:** acordar las fechas de revisión al inicio de cada ciclo, enviar previamente los puntos por resolver y registrar las decisiones. No iniciar funciones que dependan de una aprobación pendiente; continuar con tareas que ya tengan requisitos acordados.
2. **Crecimiento del alcance entre ciclos.** Las revisiones de pantallas y solicitudes pueden generar nuevas funciones que superen el presupuesto y la fecha pactados. **Mitigación:** distinguir las aclaraciones de los requisitos de las nuevas solicitudes, estimar su impacto y obtener aprobación antes de incorporarlas. Mantener fuera del alcance inicial las transferencias externas, la aplicación móvil y la aprobación automática de créditos, salvo cambio contractual autorizado.
3. **Errores de integración y regresión.** Un incremento de transferencias podría alterar las consultas ya validadas o registrar una operación de forma duplicada al comunicarse con el sistema de cuentas. **Mitigación:** comprobar la interfaz desde el primer incremento en un entorno de pruebas y repetir las pruebas de las funciones anteriores en cada ciclo. Incluir casos de interrupción, reintento y respuesta fallida; verificar que un reintento no duplique la operación y que saldos y movimientos coincidan con el sistema existente.
4. **Controles de seguridad incompletos en entregas parciales.** Al priorizar funciones visibles, el equipo podría posponer permisos o registros necesarios para proteger las consultas y transferencias. **Mitigación:** incluir requisitos y criterios de aceptación de acceso y registro de operaciones desde el primer ciclo. El responsable de seguridad deberá revisar las evidencias antes de autorizar cada entrega. Una versión mostrada para recibir comentarios no se considerará automáticamente aprobada para operar con datos o dinero reales.
5. **Documentación desactualizada.** Los ajustes de cada iteración pueden dejar diferencias entre lo implementado, los requisitos aprobados y las pruebas, dificultando la aceptación del entregable. **Mitigación:** actualizar la especificación, las interfaces, la trazabilidad y los resultados de pruebas dentro del mismo ciclo. Comprobar su correspondencia con la versión entregada y registrar la aprobación antes de cerrar el incremento.

IV. Matriz comparativa escenario por modelo

Para cada escenario se valora qué tan adecuado es cada modelo en cada criterio, según el valor que el criterio toma en ese escenario. Se usa la escala Alta (3), Media (2) y Baja (1), y el modelo con mayor total es el más adecuado para el escenario.

IV-A. Sistema de un GAD con contrato público

Tabla VI
Matriz del escenario del GAD con contrato público

Criterio	Valor en el GAD	Cascada	Iterativo	Ágil
C1. Estabilidad de los requisitos	Alta	Alta (3)	Media (2)	Baja (1)
C2. Criticidad del sistema	Media	Alta (3)	Alta (3)	Media (2)
C3. Disponibilidad del cliente	Baja	Alta (3)	Media (2)	Baja (1)
C4. Tamaño y complejidad	Grande	Media (2)	Alta (3)	Media (2)
C5. Regulación y contrato	Alta	Alta (3)	Media (2)	Baja (1)
C6. Documentación exigida	Alta	Alta (3)	Media (2)	Baja (1)
Total		**17**	**14**	**8**

IV-B. Escenario: sistema embebido de una planta industrial

Tabla VII
Matriz del escenario: controlador embebido en una línea de producción

Criterio	Valor en el escenario	Cascada	Iterativo	Ágil
C1. Estabilidad de los requisitos	Alta	Alta (3)	Media (2)	Baja (1)
C2. Criticidad del sistema	Alta	Alta (3)	Alta (3)	Media (2)
C3. Disponibilidad del cliente	Baja	Alta (3)	Media (2)	Baja (1)
C4. Tamaño y complejidad	Media	Media (2)	Alta (3)	Media (2)
C5. Regulación y contrato	Media	Alta (3)	Media (2)	Media (2)
C6. Documentación exigida	Alta	Alta (3)	Media (2)	Media (2)
Total		**17**	**14**	**10**

IV-C. Aplicación móvil de un emprendimiento

Tabla VIII
Matriz del escenario de la aplicación móvil de un emprendimiento

Criterio	Valor en la app	Cascada	Iterativo	Ágil
C1. Estabilidad de los requisitos	Baja	Baja (1)	Alta (3)	Alta (3)
C2. Criticidad del sistema	Baja	Media (2)	Media (2)	Alta (3)
C3. Disponibilidad del cliente	Alta	Media (2)	Alta (3)	Alta (3)
C4. Tamaño y complejidad	Pequeño	Media (2)	Media (2)	Alta (3)
C5. Regulación y contrato	Baja	Baja (1)	Media (2)	Alta (3)
C6. Documentación exigida	Baja	Baja (1)	Media (2)	Alta (3)
Total		**9**	**14**	**18**

La puntuación favorece al modelo ágil porque los requisitos de interfaz pueden cambiar, el propietario puede revisar avances con frecuencia y las funciones iniciales son pequeñas. El modelo iterativo también permite revisar y refinar requisitos; cascada se adapta menos a la incertidumbre de las pantallas y a la retroalimentación prevista.

IV-D. Plataforma de una cooperativa

Tabla IX
Matriz del escenario de la cooperativa

Criterio	Valor en la cooperativa	Cascada	Iterativo	Ágil
C1. Estabilidad de los requisitos	Media	Media (2)	Alta (3)	Alta (3)
C2. Criticidad del sistema	Alta	Alta (3)	Alta (3)	Media (2)
C3. Disponibilidad del cliente	Media	Alta (3)	Alta (3)	Media (2)
C4. Tamaño y complejidad	Mediano	Media (2)	Alta (3)	Alta (3)
C5. Regulación y contrato	Alta	Alta (3)	Media (2)	Media (2)
C6. Documentación exigida	Alta	Alta (3)	Alta (3)	Media (2)
Total		**16**	**17**	**14**

V. Framework de Pohl aplicado al escenario del GAD

El framework de Pohl organiza la ingeniería de requisitos en tres actividades centrales, que son la obtención, la documentación y la negociación, y dos actividades transversales, que son la validación y la gestión [9]. En el escenario del GAD se eligió el modelo en cascada, por lo que la mayor parte de las actividades centrales se concentra en la primera fase, mientras que las transversales acompañan todo el proyecto para controlar los cambios y demostrar el cumplimiento del contrato. La tabla muestra qué actividades se ejecutan en cada fase y qué artefactos producen.

Tabla X: Actividades del framework de Pohl en cada fase de la cascada para el GAD

Fase de la cascada	Actividades centrales	Actividades transversales	Artefactos
1. Requisitos	**Obtención:** Entrevistas con el administrador del contrato y los funcionarios de cada departamento, revisión de los términos de referencia y de las ordenanzas sobre trámites. **Documentación:** Redacción de la especificación de requisitos del sistema. **Negociación**: Acuerdo sobre prioridades y conflictos entre departamentos, aprobado por las autoridades del GAD.	**Validación:** Revisión formal de la especificación con el fiscalizador y los funcionarios, apoyada en prototipos de pantallas. **Gestión:** Línea base de requisitos aprobada y matriz de trazabilidad inicial.	Especificación de requisitos, prototipos de pantallas, acta de aprobación de requisitos, matriz de trazabilidad
2. Diseño	**Documentación:** Se detallan los requisitos de interfaz con los sistemas existentes del GAD. **Negociación:** Se resuelven con el GAD los requisitos que no son factibles técnicamente.	**Validación:** Se comprueba que el diseño cubre cada requisito de la línea base. **Gestión:** Se actualiza la trazabilidad de requisitos a componentes de diseño.	Documento de diseño, matriz de trazabilidad actualizada
3. Implementación	**Obtención:** Solo para aclarar dudas puntuales del equipo, sin agregar requisitos nuevos.	**Gestión:** Control de cambios, toda solicitud nueva se analiza por su impacto y se aprueba mediante una modificación formal del contrato.	Registro de solicitudes de cambio, código fuente
4. Pruebas	**Documentación:** Casos de prueba derivados de los criterios de aceptación de cada requisito.	**Validación:** Pruebas de aceptación con funcionarios y ciudadanos para confirmar que el sistema cumple lo que necesitan. **Gestión:** Se registra qué requisitos quedaron verificados.	Casos de prueba, informe de pruebas, informe de fiscalización
5. Entrega y mantenimiento	**Obtención y documentación:** Registro de nuevas necesidades para versiones futuras o un nuevo contrato.	**Validación:** Recepción del sistema con acta de entrega. **Gestión:** Archivo de la línea base final y de la trazabilidad completa para los organismos de control.	Acta de entrega y recepción, manual de usuario, línea base final

Referencias

[1] H. Washizaki, ed., *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*. IEEE Computer Society, 2024. dirección: https://www.computer.org/education/bodies-of-knowledge/software-engineering.

[2] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, 2.ª ed. Berlin, Heidelberg, Germany: Springer, 2025, ISBN: 978-3-662-69204-2. dirección: https://link.springer.com/book/9783662692042.

[3] S. Bühne, M. Glinz, H. van Loenhoud y S. Staal, *CPRE Foundation Level Syllabus*, ver. 3.3.0, International Requirements Engineering Board (IREB) e.V., Karlsruhe, Germany, 1 de abr. de 2026. dirección: https://cpre.ireb.org/en/downloads-and-resources/downloads.

[4] I. Sommerville, *Software Engineering*, 10.ª ed. Boston, MA, USA: Pearson, 2016, ISBN: 978-0-13-394303-0.

[5] E.-M. Schön, J. Thomaschewski y M. J. Escalona, «Agile Requirements Engineering: A systematic literature review,» *Computer Standards & Interfaces*, vol. 49, págs. 79-91, 2017. DOI: 10.1016/j.csi.2016.08.011.

[6] K. Schwaber, «SCRUM Development Process,» en *Business Object Design and Implementation*, J. Sutherland, C. Casanave, J. Miller, P. Patel y G. Hollowell, eds., London: Springer, 1997, págs. 117-134, ISBN: 978-1-4471-0947-1. DOI: 10.1007/978-1-4471-0947-1_11.

[7] D. Turk, R. France y B. Rumpe. «Assumptions Underlying Agile Software Development Processes.» ver. 1. arXiv: 1409.6610. (22 de sep. de 2014), dirección: https://arxiv.org/abs/1409.6610.

[8] International Electrotechnical Commission, *IEC 61508-1:2010 – Functional safety of electrical/electronic/programmable electronic safety-related systems – Part 1: General requirements*, 2.ª ed., IEC, Geneva, Switzerland, 2010, ISBN: 978-2-88910-524-3. dirección: https://webstore.iec.ch/en/publication/5515.

[9] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Berlin, Heidelberg, Germany: Springer, 2010, ISBN: 978-3-642-12577-5. DOI: 10.1007/978-3-642-12578-2.